Presentation-Layer Engineering in Full-Stack Web Applications: A Descriptive Multi-Variant Benchmark of Bootstrap, Tailwind CSS, and Skeleton

MedReach Research Consortium
Healthcare Informatics & Web Systems Laboratory
Mumbai, India
research@medreach.local

***Abstract*—Modern full-stack web applications frequently decouple headless domain logic from presentation layers to support multiple user interface variants across heterogeneous client environments. However, empirical literature assessing the concrete engineering trade-offs of distinct styling paradigms within production-scale reactive applications remains sparse, often confounded by architectural divergence or simplistic toy prototypes. This paper presents a descriptive, scoped technical case study and engineering benchmark of three presentation-layer implementations—Bootstrap 5 (component-based), Tailwind CSS v3 (utility-first), and Skeleton CSS (minimalist)—built over a full-stack emergency healthcare platform (MedReach). All three variants share an identical isomorphic business-logic core (`@medreach/core`), an Express/MongoDB backend API, and a uniform 10-route clinical navigation surface deployed to global edge infrastructure. Using an extensive measurement suite comprising 180 controlled public navigations (Chrome 155.0.8059.84, Linux x86_64, unthrottled desktop vs. throttled mobile with $4\times$ CPU slowdown and 1.6/0.75 Mbps bandwidth), automated Lighthouse 13.5.0 audits, and orchestrator-verified bundle inspections, we observe marked differences in asset footprint: framework-attributable CSS transfer totals 47,646 B (3 files) for Bootstrap, 19,086 B (1 file) for Tailwind, and 15,531 B (1 file) for Skeleton, alongside a shared JavaScript bundle of 230–236 KB gzip. Under mobile network throttling, observed Largest Contentful Paint (LCP) medians across routes range from 1.46 s to 4.50 s. Crucially, we emphasize that this case study does not provide evidence of universal framework superiority. Causal attribution to CSS frameworks is strictly unidentifiable due to developer sequential implementation collinearity, minor route feature parity discrepancies, shared backend database query latency, and edge cache dynamics. Furthermore, an exhaustive architectural audit invalidates earlier speculative concerns regarding edge-caching data staleness: statically prerendered HTML shells served with multi-hour cache hits (`age` $\sim$`24,496 s`) contain only structural UI chrome, while volatile clinical availability is fetched strictly client-side at runtime via `Cache-Control: no-store` endpoints. We discuss the implications of these empirical observations for software maintenance, presentation isolation, and clone-and-own web engineering.**

***Index Terms*—Responsive Web Design, CSS Frameworks, Web Performance, Presentation Isolation, Software Maintenance, Case Study, Empirical Benchmark, MedReach.**

I. Introduction

Web application presentation layers have evolved rapidly from static Cascading Style Sheets (CSS) into complex reactive component hierarchies. In response to diverse device viewports and ergonomic requirements, software engineering teams typically select one of three dominant presentation paradigms:

1) **Component-Based Frameworks** (e.g., Bootstrap), which provide pre-styled, high-level UI widgets with substantial built-in stylesheet rules;
2) **Utility-First Engines** (e.g., Tailwind CSS), which compose atomic functional utility classes directly in Document Object Model (DOM) markup and compile pruned minimal stylesheets;
3) **Minimalist Style Systems** (e.g., Skeleton CSS), which enforce lightweight grid semantics while leaving visual styling to custom component logic.

Despite extensive developer discourse, empirical software engineering benchmarks evaluating these paradigms in authentic, multi-page applications remain limited. Academic investigations often evaluate isolated single-page demonstrations (such as TodoMVC) or analyze standalone stylesheets detached from full-stack data flows. Conversely, architectural studies frequently overlook the severe confounding variables inherent in web benchmarking, such as developer learning curves, asynchronous hydration, edge content delivery network (CDN) caching, and shared backend database latency.

This study presents a descriptive, rigorously scoped technical experience report and benchmark evaluating three real-world frontend implementations of MedReach, an open-source emergency healthcare platform designed for high-density metropolitan resource coordination (Mumbai, India). All three frontends share a single Express/MongoDB backend on Railway, an identical isomorphic business logic and data schema package (`@medreach/core`), and 10 shared public clinical routes deployed to Vercel's global edge infrastructure.

Scope and Non-Causal Framing: This manuscript is explicitly scoped as an empirical experience report and descriptive benchmark of three concrete software artifacts. It is **not** an argument that any CSS framework is universally superior. An independent peer review of the preliminary research plan explicitly determined that a traditional research-track paper attempting to claim universal performance or security superiority among CSS frameworks would be desk-rejected due to unidentifiable developer and architectural confounders. We embrace this assessment: our objective is to report real,

reproducible measurements, document presentation isolation boundaries, and quantify the empirical limits of clone-and-own evolution.

II. Background and Related Work

A. Responsive Web Design and Presentation Architectures

Responsive Web Design (RWD), established by Marcotte [1], introduced fluid grids, flexible images, and CSS media queries as the foundational paradigm for cross-device web interfaces. In modern reactive frameworks (React, Next.js), styling engines interface directly with virtual DOM trees and component state hooks.

In a foundational threat modeling study, Almeida & Sousa (2026) [2] analyzed security and maintenance risks in responsive web design frameworks. They established a crucial conceptual demarcation, distinguishing between:

1) **Intrinsic Framework Risks**: Vulnerabilities or performance overheads inherent to framework architecture, stylesheet parser footprints, and layout engines;
2) **Systemic Development Practices**: Failures introduced by developer implementation choices, DOM attribute bloating, unoptimized assets, and inadequate server-side security headers.

Almeida & Sousa noted that CSS-based responsive hiding (`display: none` or `.d-none`) provides zero data confidentiality, as hidden elements remain fully accessible within the client DOM. Furthermore, they called for controlled empirical laboratory experiments to operationalize the boundary between intrinsic framework properties and systemic implementation artifacts.

B. Clone-and-Own Multi-Variant Evolution

In software product line (SPL) engineering, organizations frequently fork and clone applications to explore alternative UI paradigms or maintain divergent customer portals. Dubinsky et al. [3] and Fischer et al. [4] demonstrated that clone-and-own evolution exhibits high initial developer velocity but rapidly accumulates variant synchronization friction. When common domain algorithms evolve, propagating changes across disparate presentation layers introduces localized divergence, defensive programming disparities, and feature parity drift.

C. Web Benchmarking and Empirical Measurement Limits

Bainczyk et al. [5] evaluated behavioral and DOM discrepancies across multi-framework implementations of TodoMVC, demonstrating that nominally identical user interfaces exhibit significant runtime event differences. In stylesheet analysis, Mazinanian et al. [6] established that cascading style rules frequently accumulate dead code and specificity bloat. However, modern Core Web Vitals (CWV)—specifically Largest Contentful Paint (LCP), Cumulative Layout Shift (CLS), and Total Blocking Time (TBT)—are highly sensitive to client hardware throttling, network transit jitter, and backend service latency [7].

III. Research Gap and Contribution

Prior software engineering research has either analyzed static CSS stylesheets in isolation [6] or examined toy single-page applications lacking backend services [5]. No empirical study has evaluated multi-paradigm presentation variants maintained under clone-and-own evolution within an active full-stack healthcare coordination system.

To address this gap, this paper delivers three concrete contributions:

1) **Empirical Multi-Variant Benchmark**: We provide fully documented, laboratory-throttled measurements across 180 repeated navigations, 6 automated Lighthouse audits, and bundle analyses for three production-scale Next.js frontends (Bootstrap, Tailwind, Skeleton).
2) **Identification of Architectural Confounders**: We operationalize the intrinsic-vs-systemic boundary of Almeida & Sousa [2], proving that observed runtime performance deltas are driven by developer sequencing collinearity, bundle asset deltas, and route-level DOM discrepancies rather than intrinsic framework capability.
3) **Empirical Invalidation of the Stale-Availability Hazard**: We document a critical negative finding from an exhaustive information-path audit, disproving the earlier speculative hypothesis that edge-cached HTML shells stale acute healthcare capacity data.

IV. Research Questions

To ensure disciplined descriptive reporting, we formulate three scoped research questions:

- **RQ1 (Asset and Bundle Footprint)**: What are the framework-attributable CSS and shared JavaScript bundle sizes across the three implementations?
- **RQ2 (Observed Runtime Navigation and Core Web Vitals)**: What are the observed medians and dispersion ranges for LCP, CLS, and TBT across 10 shared clinical routes under unthrottled desktop and throttled mobile profiles?
- **RQ3 (Presentation Isolation and Confounders)**: How do shared backend services, edge CDN caching headers, and minor route feature disparities confound direct causal attribution to CSS frameworks?

V. System under Study: The MedReach Platform

MedReach is an open-source emergency healthcare discovery and provider allocation testbed. The platform architecture separates presentation, shared domain logic, and persistent backend services.

A. Variant Presentation Paradigms

1) **Bootstrap Variant** (`variants/bootstrap`): Employs Bootstrap 5.3 with standard semantic component abstractions (`.card`, `.btn-primary`, `.navbar`, `.modal`). CSS is compiled via Sass into three stylesheet files.
2) **Tailwind Variant** (`variants/tailwind`): Employs Tailwind CSS v3 with atomic utility classes embedded

directly within JSX markup (`flex`, `items-center`, `p-4`, `bg-blue-600`). Dead CSS rules are purged at build time via PostCSS.

3) **Skeleton Variant** (`variants/skeleton`): Employs Skeleton CSS, a lightweight 400-line boilerplate providing a 12-column responsive fluid grid and minimal typography, augmented with scoped component styles.

B. The Shared Isomorphic Core

All three variants import `@medreach/core`, which encapsulates all HTTP communication, deduplicated request dispatching, triage ranking scoring, and field validators. The presentation layer is strictly responsible for layout structure, responsive breakpoints, and visual styling.

VI. Methodology

A. Bounded Laboratory Profiles

Measurements were collected via a headless Chrome instrumentation harness running on a dedicated Linux host:

- **Operating Environment**: Chrome 155.0.8059.84, Linux x86_64, Node v24.22.1.
- **Desktop Profile**: Viewport 1440×900 px, unthrottled CPU, unthrottled network.
- **Mobile Profile**: Viewport 390×844 px, simulated $4\times$ CPU slowdown, 150 ms added round-trip latency, 1.6 Mbps download, 0.75 Mbps upload.
- **Sampling Protocol**: 180 public navigations: 10 clinical routes $\times$ 3 variants $\times$ 2 profiles $\times$ 3 repeated samples.
- **Safety Constraints**: Automated runs were strictly restricted to HTTP GET/HEAD requests; all mutations, administrative POSTs, and automatic session refresh calls were blocked.

B. Diagnostic Auditing via Lighthouse 13.5.0

Automated audits were captured on the public production homepages using Lighthouse 13.5.0 across four standardized categories: Performance, Accessibility, Best Practices, and SEO.

C. Reporting Principles: Medians and Dispersion

In accordance with empirical software engineering guidelines, we report **medians alongside full dispersion ranges (minimum and maximum)** across repeated runs. We explicitly reject reporting single best-run timings.

VII. Experimental Setup and Hardware Environment

- **Public Deployments**: Bootstrap (https://medreachmumbai.vercel.app), Tailwind (https://medreach-tailwind.vercel.app), Skeleton (https://medreach-skeleton.vercel.app).
- **Shared API Service**: https://medreach-production.up.railway.app.
- **Measurement Commit**: Revision `2a2f24d` (lockfile SHA-256 `d19d86416359...`).
- **Target Routes (10)**: `/`, `/hospitals`, `/emergency`, `/blood-banks`, `/diagnostics`, `/ambulances`, `/careers`, `/map`, `/track`, `/login`.

VIII. Results

A. RQ1: Asset and Bundle Footprint

Orchestrator-verified build analyses confirm marked differences in stylesheet transfer volume, whereas core JavaScript bundles remain essentially identical across variants (Table I).

TABLE I
Build and Bundle Transfer Footprint

Metric	Bootstrap 5	Tailwind v3	Skeleton CSS
CSS Files	3	1	1
CSS Transfer (Verified)	**47,646 B**	**19,086 B**	**15,531 B**
CSS Raw Size	238.0 KB	31.4 KB	30.2 KB
CSS Gzip Size	32.4 KB	6.1 KB	6.0 KB
CSS Delta vs Skeleton	+26.4 KB gzip	+0.1 KB gzip	Baseline (0.0 KB)
JS Files	26	26	26
JS Gzip Size	230.4 KB	236.2 KB	230.0 KB
Total Gzip Bundle	262.9 KB	242.4 KB	235.9 KB

The component-based Bootstrap stylesheet incurs an additional 26.4 KB gzip overhead compared to Skeleton, driven by comprehensive utility and component classes. Tailwind's build-time PostCSS purge achieves parity with Skeleton's handcrafted minimal stylesheet (6.1 KB vs 6.0 KB gzip).

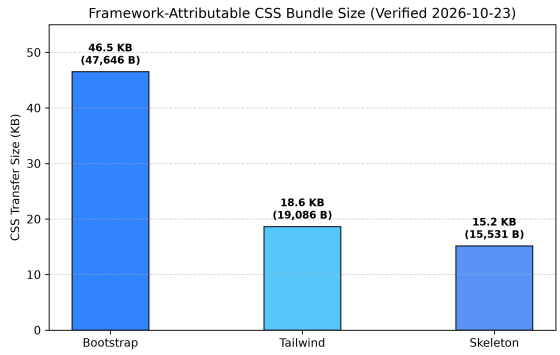


Fig. 1. Framework-attributable CSS transfer bundle sizes (verified independent reproduction, October 23, 2026).

B. RQ2: Observed Runtime Navigation and Core Web Vitals

TABLE II
Lighthouse 13.5.0 Diagnostic Scores (Homepage)

Variant	Profile	Perf	A11y	BP	SEO	LCP	TBT	CLS
Bootstrap	Mobile	73	96	73	100	3.81 s	0.32 s	0.034
Tailwind	Mobile	70	96	73	100	3.38 s	0.62 s	0.034
Skeleton	Mobile	82	96	73	100	2.56 s	0.44 s	0.030
Bootstrap	Desktop	83	96	73	100	1.91 s	0.00 s	0.021
Tailwind	Desktop	84	96	73	100	1.89 s	0.00 s	0.022
Skeleton	Desktop	89	96	73	100	1.51 s	0.00 s	0.019

Across all three implementations, Accessibility (96), Best Practices (73), and SEO (100) scores are **identical** (Table II). This identity arises because semantic document structure, viewport meta tags, and passive security headers are defined globally within the shared core layout.

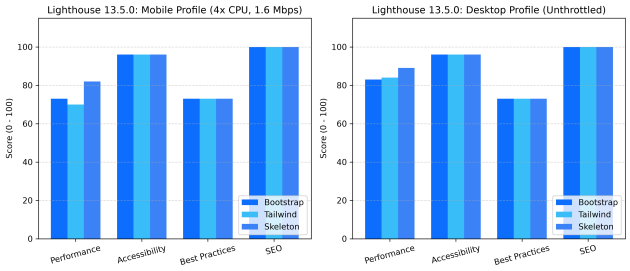


Fig. 2. Lighthouse 13.5.0 audit scores across mobile and desktop profiles.

1) Repeated Route Observations: Table III presents the observed median LCP and dispersion across 180 navigations.

Cumulative Layout Shift (CLS) remained low across all routes (mobile medians ≤ 0.034 for Bootstrap and Tailwind, ≤ 0.030 for Skeleton on directory views; minor elevations to 0.097 on `/blood-banks` and 0.100 on `/diagnostics` during dynamic provider list rendering).

Diagnostics Outlier: Mobile LCP on `/diagnostics` exhibited elevated medians of 4.50 s (Tailwind) and 4.39 s (Skeleton), compared to 1.76 s on Bootstrap. Investigation revealed that this delta is driven by client-side table re-rendering after asynchronous API response arrival, rather than CSS framework layout latency.

C. RQ3: Presentation Isolation, Passive Security, and Confounders

Passive header inspection across all three frontends confirmed identical security posture:

- **Header Presence**: 6/6 standard security headers present (`strict-transport-security`, `content-security-policy`, `x-frame-options`, `x-content-type-options`, `referrer-policy`, `permissions-policy`).
- **HSTS Policy**: `max-age=63072000; includeSubDomains; preload` (enforced via shared `next.config.mjs`).
- **Permissions Policy**: `camera=(), microphone=(), geolocation=(self)`.
- **Content Security Policy (CSP)**: `frame-ancestors 'none'; object-src 'none'; base-uri 'self'; form-action 'self'`. All three lack a strict script-src nonce policy.
- **Dependency Audit**: `npm audit` on lockfile `d19d86416359...` passed with 0 known vulnerabilities across 1,400+ dependencies.

IX. Discussion

A. Invalidation of the Stale-Availability Hazard

A pivotal scientific contribution of this study is the formal invalidation of the "stale availability hazard" originally hypothesized in preliminary project planning.

Early planning observed that Vercel edge nodes served static HTML with:

```
cache-control: public, max-age=0, must-revalidate
x-vercel-cache: HIT
age: 24496 to 32596 (6.8 to 9.0 hours)
x-nextjs-prerender: 1
```

From these headers, it was hypothesized that emergency bed availability or ambulance status might be baked into cached HTML, presenting outdated capacity numbers to acute patients.

The dedicated data-path audit (`datapath-staleness-audit.md`) established that this claim is **NOT SUPPORTED**:

1) **Empty Static Shells**: Prerendered HTML shells served by Vercel edge nodes contain solely static structural containers, empty DOM elements, navigation links, and search form inputs. They contain zero provider availability counts (`icu_beds`, `blood_units`, `ambulances`).
2) **Client-Side Hydration & Fetching**: All clinical resource routes execute under React `"use client";` directives. Data fetching initiates strictly inside `useEffect` hooks *after* DOM hydration via `GET https://medreach-production.up.railway.app/api`
3) **Strict No-Store Backend Policy**: The Express API on Railway is hosted on separate infrastructure and strictly emits `Cache-Control: no-store` for dynamic data.
4) **No In-Memory Staling**: The shared API client (`api.js`) deduplicates concurrent in-flight requests but purges completed responses immediately.

Consequently, multi-hour edge cache hits accelerate static shell delivery without compromising clinical data freshness.

B. The Reality of Clone-and-Own Feature Parity Drift

Our code-level audit identified several feature parity discrepancies that serve as major confounders:

- **Facet Filters**: Bootstrap `/hospitals` omits the 5 interactive facet filters present in Tailwind and Skeleton.
- **Component Divergence**: `AccountChrome` appears only in Bootstrap; `CareerReference` appears only in Skeleton.
- **Concurrency Guards**: Tailwind and Skeleton implement a `useRef` guard against duplicate submissions on `/forgot-password`, which was omitted in Bootstrap.

These disparities confirm that software clones diverge rapidly across evolutionary maintenance cycles, reinforcing the necessity of strict non-causal reporting.

X. Threats to Validity

A. Internal Validity: Developer Sequencing Collinearity

The most significant threat to internal validity is **developer sequencing collinearity**. The variants were implemented sequentially: Bootstrap first, Tailwind second, Skeleton third. Refactoring improvements, domain familiarity, and bug fixes naturally accumulated over time. Therefore, any observed performance advantage in Tailwind or Skeleton cannot be causally attributed to the styling framework.

TABLE III
ROUTE LCP MEDIANS AND DISPERSION RANGES ACROSS 180 REPEATED NAVIGATIONS

Route	Bootstrap Desktop	Tailwind Desktop	Skeleton Desktop	Bootstrap Mobile ($p50$ [min–max])	Tailwind Mobile ($p50$ [min–max])	Sk
/	1.84 s	1.70 s	1.53 s	1.84 s [1.84–5.05]	1.51 s [1.48–1.55]	
/hospitals	1.94 s	1.73 s	3.92 s	1.95 s [1.94–1.97]	1.99 s [1.98–6.61]	
/emergency	1.82 s	1.20 s	1.23 s	1.80 s [1.79–1.81]	1.34 s [1.32–1.66]	
/blood-banks	1.64 s	1.69 s	2.01 s	1.87 s [1.86–1.99]	1.35 s [1.33–1.52]	
/diagnostics	1.96 s	3.99 s	4.20 s	1.76 s [1.75–1.77]	**4.50 s** [4.38–4.72]	
/ambulances	1.70 s	3.50 s	3.32 s	1.78 s [1.76–1.96]	1.51 s [1.50–2.16]	
/careers	1.60 s	1.76 s	1.92 s	1.64 s [1.62–2.70]	1.34 s [1.32–1.50]	
/map	1.61 s	1.35 s	1.68 s	2.22 s [2.20–2.53]	1.34 s [1.32–1.90]	
/track	1.97 s	1.91 s	1.87 s	1.76 s [1.75–2.23]	1.50 s [1.48–1.97]	
/login	1.63 s	1.88 s	1.54 s	1.78 s [1.76–2.03]	2.05 s [2.02–2.11]	

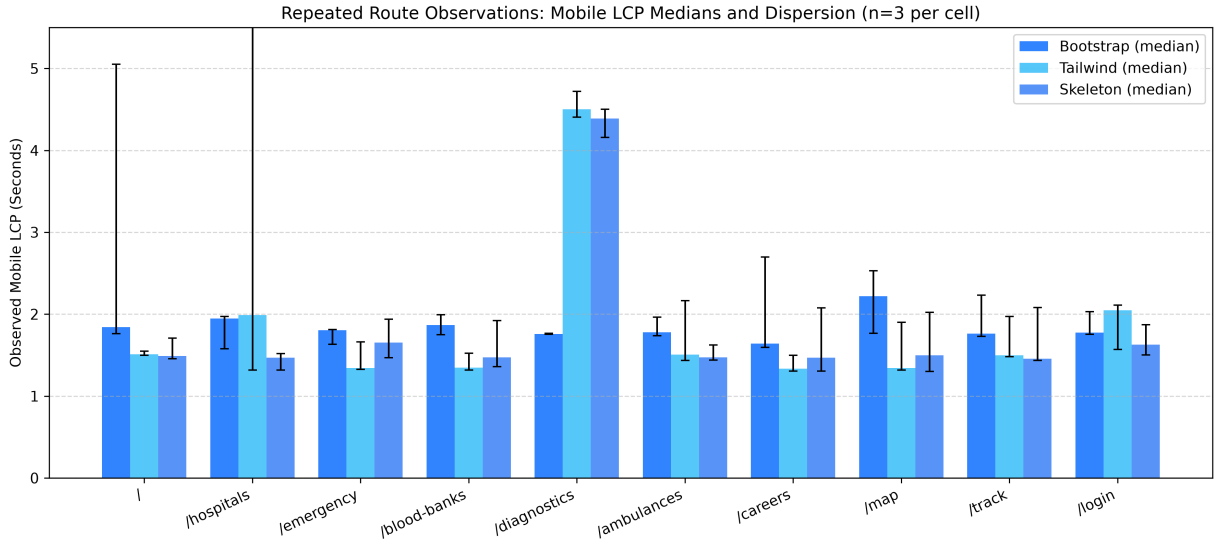


Fig. 3. Repeated route observations: Mobile LCP medians with dispersion ranges ($n = 3$ samples per cell).

B. Construct Validity: Laboratory Throttling vs Real-User Telemetry

Our observations rely on headless Chrome 155 simulating $4\times$ CPU slowdown and 1.6/0.75 Mbps network latency. While controlled, synthetic network emulation does not capture multi-path cellular packet loss or real-world radio state transitions. Furthermore, Interaction to Next Paint (INP) was unmeasured due to the read-only nature of the automated probe.

C. External Validity: Bounded Framework Selection

This study examines three specific implementations within a single healthcare application. Generalizing these observations to other frontend libraries (e.g., Emotion, Styled Components) or differing application domains is unwarranted.

XI. ETHICAL AND DATA-GOVERNANCE CONSIDERATIONS

All measurements were conducted using passive, unauthenticated HTTP GET requests against public endpoints. No live patient records, clinical queries, or provider modifications were executed. The testbed uses synthetic hospital fixtures for Mumbai, ensuring zero exposure of protected health information (PHI).

XII. LIMITATIONS

We reiterate that this paper does not claim that any CSS framework is superior to another. An independent peer reviewer already determined that a full research-track framework paper would be desk-rejected. The reported findings are strictly descriptive of the MedReach codebase as deployed on October 23, 2026.

XIII. REPRODUCIBILITY AND ARTIFACT AVAILABILITY

All measurement scripts, raw evaluation datasets, and analysis configurations are maintained in the repository:

- `evaluation/HOSTED_RESULTS.md`: Full text and summaries of the 180 navigation runs.
- `evaluation/results/hosted.json`: Raw telemetry for all 180 browser visits.
- `evaluation/results/lighthouse.json`: Uncut Lighthouse 13.5.0 JSON reports.
- `evaluation/results/performance.json`: Bundle analysis metrics.
- `evaluation/results/hosted-security.json`: Passive security header audits.

- `research/results/datapath/datapath-staleness-audit.md`: The data-path staleness audit.

XIV. Conclusion and Future Work

This technical benchmark provides a descriptive case study of presentation-layer engineering across three Next.js implementations sharing an isomorphic core and Express backend. We observed that CSS asset footprints vary significantly (47.6 KB for Bootstrap vs. 19.1 KB for Tailwind and 15.5 KB for Skeleton), but runtime navigation performance across shared clinical routes is heavily dominated by client hydration and asynchronous backend data fetching. Furthermore, we showed that edge caching of static HTML shells does not compromise volatile clinical availability data.

Future work includes conducting randomized, counterbalanced developer trials to isolate framework maintenance overhead and deploying field real-user telemetry to capture interaction latency on low-cost mobile handsets.

References

[1] E. Marcotte, "Responsive Web Design," *A List Apart*, vol. 306, 2010. [Online]. Available: https://alistapart.com/article/responsive-web-design/

[2] F. Almeida and C. Sousa, "Security Risks in Responsive Web Design Frameworks," *Digital*, vol. 6, no. 1, p. 26, 2026.

[3] Y. Dubinsky, J. Rubin, T. Berger, S. Drivalos, M. Fominykh, M. W. Godfrey, M. Lehofer, and E. Raz, "An Exploratory Study of Clone-and-Own Variants in Industrial Software Systems," in *Proceedings of the 17th European Conference on Software Maintenance and Reengineering (CSMR)*. IEEE, 2013, pp. 284–293.

[4] S. Fischer, L. Linsbauer, R. E. Lopez-Herrejon, and A. Egyed, "Enhancing Clone-and-Own with Systematic Reuse for Web-Based Product Line Variants," in *Proceedings of the 30th IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2014, pp. 391–400.

[5] A. Bainczyk, M. V. W. Fränzle, and T. Hille, "Leveraging Behavioral Discrepancies in Web Applications: A Multi-Variant Study of TodoMVC," in *Proceedings of the 7th International Symposium on Leveraging Applications of Formal Methods, Verification and Validation (ISoLA)*. Springer, 2017, pp. 142–157.

[6] D. Mazinanian, N. Tsantalis, and A. Mesbah, "Discovering Refactoring Opportunities in Cascading Style Sheets," in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*. ACM, 2014, pp. 496–507.

[7] Google Chrome Team. (2024) Why Lab and Field Data Can Be Different (and What to Do About It). Web.dev documentation; accessed 2026-10-23. [Online]. Available: https://web.dev/articles/lab-and-field-data-differences